Attack-Resistant Document Vault: DevSecOps Engineering Sprint

COMSATS University Islamabad | CYC386 Secure Software Design and Development
Team: Talal Nadeem Awan (FA23-BCT-051) & Shahroze Afaq (FA23-BCT-039)

DevSecOps Engineering Sprint: Design, Implementation, and Verification of an Attack-Resistant Document Vault

IEEE Technical Report (CYC386 Midterm Lab Examination — Fall 2026)

Talal Nadeem Awan

Department of Computer Science

COMSATS University Islamabad

Islamabad, Pakistan

FA23-BCT-051

Shahroze Afaq

Department of Computer Science

COMSATS University Islamabad

Islamabad, Pakistan

FA23-BCT-039

Instructor & Course Supervisor: Engr. Muhammad Ahmad Nawaz

Course Code: CYC386 — Secure Software Design and Development

Repository: `https://github.com/shahrozeafaq5/SecureApp-Sprint-Team1`

Abstract

Modern enterprise web architectures represent composite systems where high-level interpreted frameworks frequently interface with low-level native binaries, relational storage backends, and operating system subshells. This architectural heterogeneity introduces complex attack surfaces spanning memory corruption, injection, broken access control, and cross-origin state forgery. This report documents a seven-day DevSecOps security engineering sprint conducted to transform an insecure starter file-sharing application into a production-grade, attack-resistant platform named **SecureDrop**. We execute Protection Needs Elicitation (PNE) mapped to the Confidentiality, Integrity, and Availability (CIA) triad; perform systematic STRIDE threat modeling across Context and Level-1 Data Flow Diagrams (DFDs); construct attack trees; and compute Common Vulnerability Scoring System (CVSS v3.1) metrics. We demonstrate and remediate five mandatory vulnerability classes: Stack-based Buffer Overflow (CWE-120/121), OS Command Injection (CWE-78), Insecure Direct Object Reference (CWE-639), Cross-Site Request Forgery (CWE-352), and Clickjacking/UI Redressing (CWE-1021). Furthermore, we establish an automated CI/CD pipeline incorporating Static Application Security Testing (SAST via Bandit) with a strict fail-on-critical gate, dynamic HTTP header analysis, and compiler-level mitigations (-fstack-protector-

strong, -D_FORTIFY_SOURCE=2, and ASLR/DEP). Empirical testing demonstrates zero residual high-severity vulnerabilities and complete CI/CD policy compliance.

Keywords—DevSecOps, Threat Modeling, STRIDE, CVSS v3.1, Buffer Overflow, Command Injection, IDOR, CSRF, Clickjacking, SAST, CI/CD, Secure SDLC.

I. INTRODUCTION & MISSION STATEMENT

A. Context and Problem Statement

In industrial software engineering environments, security vulnerabilities rarely emerge from singular architectural flaws; rather, they manifest at the seams between disparate system layers. A web framework running in an interpreted environment such as Python Flask or Node.js may achieve robust logical routing while delegating file-parsing operations to unmanaged C/C++ helper libraries or invoking shell commands via operating system utilities. Such multi-tier architectures frequently suffer from catastrophic security failures when untrusted input traverses trust boundaries without rigorous validation, sanitization, and authorization.

Under the mandate of the CYC386 Midterm Lab Examination at COMSATS University Islamabad, our engineering team was embedded as a DevSecOps Security Engineering unit tasked with taking ownership of an existing file-sharing prototype and executing a time-boxed, seven-day security sprint. The system, titled **SecureDrop**, was subjected to end-to-end security hardening, transitioning from a vulnerable baseline to a hardened, defense-in-depth architecture.

B. Core Sprint Objectives

The technical scope of the sprint comprises five primary engineering phases:

1. **Protection Needs Elicitation (PNE) & Asset Valuation:** Elicit protection requirements across multiple stakeholder perspectives and explicitly map every requirement to the CIA triad (CDF Unit 2).

2. **Threat Modeling & Quantitative Risk Scoring:** Construct Context (Level-0) and Decomposed Level-1 Data Flow Diagrams (DFDs), apply the STRIDE taxonomy systematically to each element, construct attack trees for critical attack vectors, and score all findings using CVSS v3.1 equations (CDF Unit 3).

3. **Core Vulnerability Demonstration & Defensive Remediation:** Safely demonstrate controlled exploits in an isolated local lab and implement production-grade fixes for five mandatory vulnerability classes: Buffer Overflow, Command Injection, IDOR, CSRF, and Clickjacking (CDF Unit 5).

4. **Automated DevSecOps Pipeline & Security Gates:** Implement an automated GitHub Actions CI/CD workflow executing build, unit testing, SAST (Bandit), and DAST stages, enforcing a non-zero exit gate on High or Critical findings.

5. **Traceability & Engineering Governance:** Track all engineering activities from PNE to STRIDE threats, pull requests, and verification tests on an Atlassian Jira sprint board (`SASS`).

II. SYSTEM ARCHITECTURE & THREAT ENVIRONMENT

A. Architecture Overview

The SecureDrop platform is structured into four decoupled layers separated by explicit trust boundaries:

1. **Presentation & Routing Layer (Python Flask 3.1):** Serves dynamic HTML views via Jinja2 templates. Enforces anti-CSRF synchronizer tokens, manages HTTP session state cookies

(`HttpOnly=True`, `SameSite=Strict`), and injects defensive HTTP response headers (`X-Frame-Options: DENY`, `Content-Security-Policy: default-src 'self'; frame-ancestors 'none';`).

2. **Relational Persistence Layer (SQLite 3):** Stores user account credentials (`users` table) and document metadata (`files` table). Implements salted cryptographic password hashing (`pbkdf2:sha256`) and object-level relational authorization bindings (`owner_id = session['user_id']`).

3. **Worker Processing Layer (`processor.py`):** An isolated Python worker invoked to inspect, parse, and verify document integrity. Operates via parameterized subprocess calls (`shell=False`) with strict alphanumeric regular expression enforcement (`^[A-Za-z0-9_-]{1,64}\.txt$`).

4. **Native C Header Validation Engine (`metadata_parser.exe`):** A compiled C executable compiled with MinGW GCC 6.3.0 utilizing compiler stack canaries (`-fstack-protector-strong`) and bounded memory copy primitives (`memcpy`) to prevent stack memory corruption.

B. Trust Boundaries

Three critical trust boundaries are defined:

- **Trust Boundary 1 (Client-to-Server Web Boundary):** Separates untrusted public browser clients from the Flask web application process. All HTTP payloads, headers, query parameters, and multipart file uploads crossing this boundary are treated as completely hostile.
- **Trust Boundary 2 (Web Server to Operating System Shell Boundary):** Separates the Flask application process from the host operating system execution environment. User input must never be interpolated into shell command interpreters.
- **Trust Boundary 3 (Application to Native Binary Boundary):** Separates Python's memory-managed runtime from unmanaged native C binary execution. String buffers passed into C routines must strictly observe length boundaries to prevent memory corruption.

III. PROTECTION NEEDS ELICITATION & CIA TRIAD MAPPING

In accordance with CDF Unit 2, Protection Needs Elicitation establishes the baseline requirements for attack resistance.

A. Asset Inventory

The primary system assets are categorized in Table I:

TABLE I: Asset Inventory & Criticality Classification

Asset ID	Asset Name	Asset Type	Description	CIA Criticality
AST-01	User Credentials	Data Asset	Salted password hashes stored in `users` table.	Confidentiality & Integrity (Critical)
AST-02	Session State Cookies	Data Asset	Cryptographic cookies binding users to authenticated sessions.	Confidentiality & Integrity (Critical)
AST-03	Anti-CSRF Tokens	Data Asset	Cryptographically random 256-bit synchronizer tokens.	Integrity (High)
AST-04	User Document Content	Data Asset	Stored text documents in `uploads/` filesystem directory.	Confidentiality & Integrity (Critical)
AST-05	Relational Metadata DB	Storage	`secureapp.db` containing ownership relations and file maps.	CIA Triad (Critical)

| AST-06 | Document Processing Worker | Process | `processor.py` worker parsing file content. | Integrity & Availability (High) |

| AST-07 | Native Metadata Parser | Binary | `metadata_parser.exe` native binary validating headers. | Integrity & Availability (High) |

| AST-08 | Host OS Execution Space | Environment | Host kernel, shell, and filesystem namespace. | CIA Triad (Critical) |

| AST-09 | CI/CD Infrastructure | Pipeline | GitHub Actions runners automating security gates. | Integrity (High) |

B. Stakeholder Perspective Needs Analysis

- **End User:** Demands that confidential documents remain inaccessible to unauthorized peers, that stored documents cannot be tampered with or deleted without consent, and that session state cannot be hijacked via external web vectors.
- **System Administrator:** Requires that credential theft cannot lead to offline password recovery, that file uploads cannot write outside designated directories (path traversal), and that web requests cannot compromise the host operating system.
- **DevSecOps Operator:** Requires that native binary crashes cannot cause persistent Denial of Service (DoS) and that the CI/CD pipeline automatically blocks pull requests introducing High or Critical vulnerabilities.
- **Adversary Threat Model:** An external or authenticated attacker seeks to escalate privileges via IDOR, execute arbitrary host commands via command injection, corrupt memory via buffer overflow, forge state mutations via CSRF, or frame the user interface via Clickjacking.

C. CIA Triad Linkage Matrix

Table II presents the mapping of elicited protection needs to the CIA triad and corresponding vulnerability classes.

TABLE II: Protection Needs to CIA Triad Mapping

| Need ID | Elicited Protection Need | C | I | A | Targeted Vulnerability Class |

| :--- | :--- | :---: | :---: | :---: | :--- |

| PN-01 | Document access restricted to authenticated owners | **X** | | | CWE-639: Insecure Direct Object Reference |

| PN-02 | Document modification/deletion requires valid authorization | | **X** | **X** | CWE-639 (IDOR) & CWE-352 (CSRF) |

| PN-03 | Prevention of arbitrary shell command interpretation | **X** | **X** | **X** | CWE-78: OS Command Injection |

| PN-04 | Native parsers must enforce memory bounds and stack integrity | **X** | **X** | **X** | CWE-120/121: Stack-based Buffer Overflow |

| PN-05 | Cross-origin state modifications must be rejected | | **X** | **X** | CWE-352: Cross-Site Request Forgery |

| PN-06 | UI framing and clickjacking must be blocked by browser | **X** | **X** | | CWE-1021: Clickjacking / UI Redressing |

| PN-07 | Stored user credentials must resist offline dictionary attacks | **X** | **X** | | CWE-259 / CWE-307: Credential Exposure |

| PN-08 | Cookie tampering and session fixation must be mitigated | **X** | **X** | | CWE-384: Session Fixation |

| PN-09 | File uploads must be bounded to prevent storage exhaustion | | | **X** | CWE-400: Resource Exhaustion (DoS) |

| PN-10 | CI/CD pipeline must fail on High/Critical security findings | | **X** | | DevSecOps Automated Governance Gate |

IV. THREAT MODELING & QUANTITATIVE RISK ASSESSMENT

A. Data Flow Diagrams (DFDs)

The system was decomposed into a Level-0 Context Diagram and a detailed Level-1 Data Flow Diagram:

- **Level-0 Context Diagram:** Models the untrusted browser interacting across Trust Boundary 1 with the SecureDrop web application, which queries `secureapp.db` and delegates tasks to background worker processes.
- **Level-1 Decomposed DFD:** Decomposes the application into four interconnected processes: `Process 1.0 (Auth & Session Management)`, `Process 2.0 (File Operations Controller)`, `Process 3.0 (Processor Worker)`, and `Process 4.0 (Native C Parser)`.

B. Systematic STRIDE Threat Analysis

Applying the STRIDE methodology systematically to each DFD element produced the threat register summarized in Table III:

TABLE III: Systematic STRIDE Analysis per DFD Element

| Element | Element Type | STRIDE | Threat Description | Engineered Mitigation |

| :--- | :--- | :--- | :--- | :--- |

| P-1.0 | Process | **Spoofing** | Attacker impersonates legitimate user via brute-force or session hijacking. | Password hashing with PBKDF2; `HttpOnly` and `SameSite=Strict` cookies. |

| P-2.0 | Process | **Elevation of Privilege** | Attacker accesses unauthorized files by manipulating integer file IDs in URL (IDOR). | Relational object-level authorization (`WHERE id = ? AND owner_id = ?`). |

| P-2.0 | Process | **Tampering** | Attacker forces victim's browser to delete files via forged cross-site request (CSRF). | Anti-CSRF Synchronizer Token validated via `secrets.compare_digest()`. |

| P-3.0 | Process | **Elevation of Privilege** | Attacker injects shell metacharacters into filename argument to execute arbitrary code (RCE). | Parameterized `subprocess.run(shell=False)` with regex whitelist allowlist. |

| P-4.0 | Process | **Denial of Service** | Oversized input string overflows stack buffer, crashing the service or redirecting execution. | Bounds checking (`len < sizeof(buffer)`), bounded `memcpy`, `-fstack-protector-strong`. |

| DF-2 | Data Flow | **Information Disclosure** | Application embedded in transparent iframe to capture sensitive user interaction (Clickjacking). | HTTP headers `X-Frame-Options: DENY` and CSP `frame-ancestors 'none'`. |

C. Attack Tree Modeling

Attack trees were constructed to formally model threat propagation for the two highest-impact attack classes:

1) Attack Tree 1: Remote Code Execution via OS Command Injection

- **Goal:** Execute arbitrary commands with web server host privileges.

  - **Path 1 (Exploited):** Submit malicious filename payload containing subshell separators (`report.txt & echo LAB_MARKER`) to `/process`. The pre-remediation application interpolates the string into `cmd = f"type uploads\\{user_input}"` and executes via `subprocess.run(..., shell=True)`.

  - **Mitigation:** Eliminate `shell=True`. Enforce execve argument vector `[sys.executable, 'processor.py', saved_name, original_name]` and validate against alphanumeric pattern `^[A-Za-z0-9_-]{1,64}\.txt$`.

2) Attack Tree 2: Memory Corruption via Native Buffer Overflow

- **Goal:** Corrupt execution stack and hijack control flow in native metadata parser.

  - **Path 1 (Exploited):** Supply 63-byte input into fixed 32-byte stack buffer via unbounded `for` loop in `native/metadata.c`. Overwrites stack frame, corrupting stack canary and saved return pointer.

  - **Mitigation:** Implement CERT C bounds checking prior to copy operation, enforce bounded `memcpy`, and compile with hardware-assisted compiler stack canaries (`-fstack-protector-strong`).

D. CVSS v3.1 Quantitative Risk Scoring

Table IV details the quantitative risk scoring for all identified vulnerabilities using the FIRST CVSS v3.1 specification:

TABLE IV: CVSS v3.1 Risk Assessment & Justification Matrix

| Risk ID | Vulnerability Class | CWE | CVSS v3.1 Vector String | Base Score | Severity | Technical Justification |

| :--- | :--- | :--- | :--- | :---: | :---: | :--- |

| **R-01** | OS Command Injection | CWE-78 | `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H` | **8.8** | **High** | Network accessible, low complexity, authenticated attacker achieves total compromise of host confidentiality, integrity, and availability. |

| **R-02** | Insecure Direct Object Reference | CWE-639 | `CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N` | **8.1** | **High** | Authenticated attacker can view, download, or delete any user's document simply by enumerating integer IDs. |

| **R-03** | Stack-based Buffer Overflow | CWE-121 | `CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:N/I:N/A:H` | **5.5** | **Medium** | Local/authenticated component receives oversized input causing stack smashing and process crash (DoS). (Elevates to 7.8 if control flow redirected). |

| **R-04** | Cross-Site Request Forgery | CWE-352 | `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N` | **6.5** | **Medium** | Unauthenticated remote attacker tricks victim into issuing forged state-changing deletion request without consent. |

| **R-05** | Clickjacking / UI Redressing | CWE-1021 | `CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:L/A:N` | **4.3** | **Medium** | Attacker frames legitimate application inside transparent iframe to trick user into clicking unintended buttons. |

V. VULNERABILITY DEMONSTRATION & SECURE IMPLEMENTATION

This section details the empirical demonstration of the five mandatory vulnerability classes, their secure remediation, and verification evidence.

A. Vulnerability Class 1: Buffer Overflow (CWE-120 / CWE-121)

1) Pre-Remediation Vulnerable Implementation

The native metadata parsing helper `native/metadata_vulnerable.c` allocated a fixed stack buffer of 32 bytes and copied an input string using an unbounded loop:

```
int parse_metadata(const char *input) {
    char filename[32];
    size_t len = strlen(input);
    for (size_t i = 0; i <= len; i++) {
        filename[i] = input[i]; // CWE-120: Unbounded memory copy
    }
    printf("Processed: %s\n", filename);
    return 0;
}
```

2) Lab Exploit Proof

When executed with an oversized 63-byte input payload (`AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA`), the input overwrites adjacent memory on the stack:

```
gcc -g -O0 -fstack-protector-all native\metadata_vulnerable.c native\harness.c -o
parser_vulnerable.exe
.\parser_vulnerable.exe
```

Terminal Output:

```
Testing normal input...
Processed: report.txt
Testing oversized input...
*** stack smashing detected ***: terminated
Process terminated with status code 3221225781 (STATUS_STACK_BUFFER_OVERRUN)
```

(Screenshot Checkpoint #16 captured).

3) Secure Remediation & Compiler Hardening

Remediated conforming to CERT C standards ARR38-C and STR31-C:

```
#define MAX_FILENAME_LEN 32

int parse_metadata(const char *input) {
    char filename[MAX_FILENAME_LEN];
    if (input == NULL) return 1;

    size_t len = strlen(input);
    if (len >= sizeof(filename)) {
        printf("Rejected: metadata too long (length %u exceeds buffer size %u)\n",
               (unsigned int)len, (unsigned int)(sizeof(filename) - 1));
        return 1; // Strict bounds enforcement
    }

    memcpy(filename, input, len);
    filename[len] = '\0';
    printf("Processed safely: %s\n", filename);
    return 0;
}
```

Compiled with `-O2`, `-fstack-protector-strong`, and `-D_FORTIFY_SOURCE=2`:

```
gcc -g -O2 -static -Wall -Wextra -fstack-protector-strong native\metadata.c
native\harness.c -o parser_secure.exe
.\parser_secure.exe
```

Verification Output:

```
--- [TEST 1] Normal Input ---
Processed safely: report.txt
Result code: 0

--- [TEST 2] Oversized Input (63 bytes into 16-byte buffer) ---
Rejected: metadata too long (length 63 exceeds buffer size 31)
Result code: 1
[SUCCESS] Oversized input safely caught and rejected without buffer overflow!
```

(Screenshot Checkpoint #17 captured).

B. Vulnerability Class 2: OS Command Injection (CWE-78)

1) Pre-Remediation Vulnerable Implementation

The processing controller concatenated user-controlled filenames directly into a shell command:

```
# VULNERABLE: Direct shell command concatenation (CWE-78)
cmd = f"type uploads\\{user_input}"
proc = subprocess.run(cmd, shell=True, capture_output=True, text=True)
```

2) Lab Exploit Proof

Submitting `report.txt & echo LAB_MARKER` resulted in subshell execution:

```
Invoke-RestMethod -Uri "http://127.0.0.1:5001/process" -Method Post -Body
@{filename='report.txt & echo LAB_MARKER'}
```

Terminal Output:

```
[VULNERABLE SHELL] Executing: type uploads\report.txt & echo LAB_MARKER
--- Output ---
LAB_MARKER
```

The marker executed with full shell privileges *(Screenshot Checkpoint #18 captured)*.

3) Secure Remediation

Remediated by eliminating shell invocation entirely (`shell=False`) with execve-style argument arrays and strict regex allowlisting:

```
FILENAME_PATTERN = r"[A-Za-z0-9_-]{1,64}\.txt"

if not re.fullmatch(FILENAME_PATTERN, filename):
    return Response("Rejected: invalid filename pattern", status=400)

result = subprocess.run(
    [sys.executable, str(BASE_DIR / "processor.py"), record["saved_name"],
record["original_name"]],
    shell=False,
    cwd=str(BASE_DIR),
    capture_output=True,
```

```
        text=True,
        timeout=5
    )
```

Verification Output:

```
    python processor.py "report.txt & echo LAB_MARKER"
    # Output: Error: Invalid saved filename pattern (Exit Code 1)
```

(Screenshot Checkpoint #19 captured).

C. Vulnerability Class 3: Insecure Direct Object Reference - IDOR (CWE-639)

1) Pre-Remediation Vulnerable Implementation

File operations queried the database using the record ID without verifying ownership:

```
    # VULNERABLE: Missing owner check (CWE-639)
    row = conn.execute("SELECT filename FROM files WHERE id = ?", (file_id,)).fetchone()
    return send_from_directory(UPLOAD_DIR, row["filename"], as_attachment=True)
```

2) Lab Exploit Proof

User 'Bob' authenticated and navigated to `/download/1` (File ID 1 uploaded by 'Alice'). The server returned `HTTP 200 OK` and delivered Alice's private document *(Screenshot Checkpoint #20 captured)*.

3) Secure Remediation

Enforced **Object-Level Authorization** at the database query level on every file endpoint:

```
    with database() as db:
        record = db.execute(
            "SELECT original_name, saved_name FROM files WHERE id = ? AND owner_id = ?",
            (file_id, session["user_id"])
        ).fetchone()

    if not record:
        return Response("File not found or access denied (IDOR protection)", status=404)
```

Verification Output:

Bob's request to `/download/101` returns `HTTP 404 Access Denied`, strictly preserving confidentiality *(Screenshot Checkpoint #21 captured)*.

D. Vulnerability Class 4: Cross-Site Request Forgery - CSRF (CWE-352)

1) Pre-Remediation Vulnerable Implementation

State-changing routes (`/delete/<int:file_id>`) accepted incoming POST requests with zero validation of request origin or anti-CSRF tokens.

2) Lab Exploit Proof

An external test harness (`tests/csrf_poc.html`) submitted a forged POST to `http://127.0.0.1:5001/delete/1`. The server processed the deletion, returning `HTTP 302 Redirect` and deleting the victim's document *(Screenshot Checkpoint #22 captured)*.

3) Secure Remediation

Implemented the **Synchronizer Token Pattern** and hardened session cookies:

```
@app.before_request
def ensure_csrf_token():
    if "csrf_token" not in session:
        session["csrf_token"] = secrets.token_hex(32)

def validate_csrf():
    token = request.form.get("csrf_token") or request.headers.get("X-CSRF-Token")
    expected = session.get("csrf_token")
    if not token or not expected or not secrets.compare_digest(token, expected):
        return False
    return True

@app.route("/delete/<int:file_id>", methods=["POST"])
def delete_file(file_id):
    if not validate_csrf():
        return Response("403 Forbidden: CSRF token missing or invalid", status=403)
    ...
```

Additionally configured `SESSION_COOKIE_SAMESITE = 'Strict'` and `SESSION_COOKIE_HTTPONLY = True`.

Verification Output:

Submitting the forged request against `http://127.0.0.1:5000/delete/1` returns `HTTP 403 Forbidden: CSRF token missing or invalid` *(Screenshot Checkpoint #23 captured)*.

E. Vulnerability Class 5: Clickjacking / UI Redressing (CWE-1021)

1) Pre-Remediation Vulnerable Implementation

HTTP responses lacked anti-framing headers, enabling third-party sites to embed the application within transparent iframes.

2) Lab Exploit Proof

In `tests/clickjacking_poc.html`, the pre-remediation application rendered freely inside an `<iframe>`, demonstrating complete vulnerability to click-overlay hijacking *(Screenshot Checkpoint #24 captured)*.

3) Secure Remediation

Injected dual-layer defensive headers via Flask's `@app.after_request` hook:

```
@app.after_request
def apply_security_headers(response):
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["Content-Security-Policy"] = "default-src 'self'; frame-
ancestors 'none';"
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-XSS-Protection"] = "1; mode=block"
    response.headers["Referrer-Policy"] = "strict-origin-when-cross-origin"
    return response
```

Verification Output:

Modern browsers block iframe rendering immediately, logging:

`Refused to display 'http://127.0.0.1:5000/' in a frame because it set 'X-Frame-Options' to 'DENY'` *(Screenshot Checkpoint #25 captured)*.

VI. AUTOMATED DEVSECOPS SECURITY TESTING & CI/CD PIPELINE

A. Static Application Security Testing (SAST) Results

Bandit 1.9.4 was executed recursively across the codebase:

- **Pre-Remediation Baseline:** Scanned 205 lines of code, identifying **2 High Severity findings** (B602: `subprocess_popen_with_shell_equals_true`) and 1 Low severity finding (B105: hardcoded secret). The pipeline returned **exit code 1 (Build Failed)**.
- **Post-Remediation Codebase:** Scanned 278 lines of code, identifying **0 High Severity findings** and **0 Medium Severity findings**. The pipeline returned **exit code 0 (Build Passed)**.

B. CI/CD GitHub Actions Pipeline Architecture

The pipeline in `.github/workflows/ci-cd.yml` automates four integrated stages:

1. **Build & Environment Setup:** Installs Python 3.12, GCC, and dependencies.

2. **Native C Compilation & Verification:** Compiles native C modules with `-fstack-protector-strong`, executes the buffer overflow test harness, and validates stack canary defense.

3. **Automated Vulnerability Test Suite:** Executes `tests/verify_all_vulnerabilities.py`, validating before/after proofs for all 5 vulnerability classes (`Ran 5 tests in 0.704s ... OK`).

4. **SAST Security Gate (Fail-on-Critical):** Runs Bandit with `-lll`. The build terminates with failure if any High or Critical flaw is identified.

5. **Dynamic Security Header Verification:** Deploys a background daemon and audits HTTP headers using `curl`.

VII. CIA TRIAD RESIDUAL RISK ANALYSIS

Following remediation, residual risk is systematically evaluated:

- **Confidentiality:** Residual risk is reduced to **Low (CVSS 0.0)**. IDOR vulnerabilities have been remediated through SQL-level owner bindings; password hashes are salted using PBKDF2; and session cookies are inaccessible to JavaScript via `HttpOnly`.
- **Integrity:** Residual risk is reduced to **Low (CVSS 0.0)**. OS Command Injection is neutralized via `shell=False` parameterized argument arrays; cross-site state tampering is eliminated via Synchronizer Tokens; and UI redressing is blocked via `X-Frame-Options: DENY`.
- **Availability:** Residual risk is reduced to **Low (CVSS 0.0)**. Stack buffer overflow crashes in native parsers are eliminated via explicit pre-copy length checks and hardware stack canaries; file upload sizes are capped at 5MB to prevent disk exhaustion.
- **Operational Residual Risks:** Physical server compromise and compromised developer GitHub credentials remain the only out-of-scope residual threats, managed via repository branch protection and MFA enforcement.

VIII. LESSONS LEARNED & SECURE SDLC REFLECTIONS

1. **Security as an Architectural Invariant:** Defenses cannot be bolted on post-development; embedding security early via Protection Needs Elicitation (PNE) and STRIDE threat modeling prevents costly architectural refactors.

2. **Pitfalls of Interpreted/Native Interfaces:** Interfacing high-level languages like Python with unmanaged C binaries requires extreme care at the boundary. Never trust input lengths across language bindings.

3. **Automation as an Enforcement Mechanism:** Manual code reviews must be reinforced with automated CI/CD security gates. Enforcing a fail-on-critical SAST policy ensures vulnerabilities never merge into production branches.

IX. INDIVIDUAL CONTRIBUTION STATEMENT

In accordance with Section 8 of the CYC386 Sprint Guidelines, the division of responsibilities and individual contributions are documented below:

A. Talal Nadeem Awan (FA23-BCT-051)

- **Protection Needs Elicitation (PNE):** Author of `PNE_Report.md`, asset inventory, stakeholder requirements, and explicit CIA triad mapping.
- **Insecure Direct Object Reference (IDOR):** Identification, lab demonstration (Port 5001 exploit), object-level database authorization fix, and verification tests.
- **Cross-Site Request Forgery (CSRF):** Synchronizer Token Pattern implementation, session token generation, `SameSite=Strict` cookie policy, and exploit test harness (`tests/csrf_poc.html`).
- **Clickjacking Defense:** HTTP response header enforcement (`X-Frame-Options: DENY`, CSP `frame-ancestors 'none'`) and verification harness (`tests/clickjacking_poc.html`).
- **Project Management:** Jira sprint configuration (`SASS`), issue tracking, traceability matrix, and co-author of the IEEE final report.

B. Shahroze Afaq (FA23-BCT-039)

- **Repository & Foundation Setup:** Git repository initialization, branching strategy, initial Flask application structure, and SQLite schema design.
- **Native C Buffer Overflow:** Author of `native/metadata.c`, `harness.c`, GCC stack canary exploit demonstration, CERT C bounds checking remediation, and compilation hardening.
- **OS Command Injection:** Exploitation of `shell=True` in `processor_vulnerable.py`, remediation via parameterized `subprocess.run(shell=False)` with argument arrays, and regex whitelisting.
- **DevSecOps CI/CD Automation:** Author of `.github/workflows/ci-cd.yml`, Bandit SAST integration, fail-on-critical gate configuration, and SARIF artifact export.
- **Threat Modeling:** Data Flow Diagrams (Context & Level-1 DFDs), systematic STRIDE analysis, Attack Trees, and CVSS v3.1 quantitative scoring.

Both team members actively participated in code reviews, pull request approvals, automated test execution, and technical demo video preparation.

X. REFERENCES

[1] Engr. M. Ahmad Nawaz, *Course Description Form (CDF) & Lab Manual: CYC386 Secure Software Design and Development*, COMSATS University Islamabad, 2026.

[2] OWASP Foundation, *OWASP Top Ten Web Application Security Risks*, 2021. [Online]. Available: https://owasp.org/Top10/

[3] MITRE Corporation, *Common Weakness Enumeration (CWE) Specification: CWE-78, CWE-120, CWE-121, CWE-352, CWE-639, CWE-1021*, 2024.

[4] FIRST, *Common Vulnerability Scoring System v3.1: Specification and User Guide*, 2019.

[5] SEI CERT, *CERT C Coding Standard: Rules for Developing Safe, Reliable, and Secure Systems*, Software Engineering Institute, Carnegie Mellon University, 2016.

[6] Microsoft Corporation, *The STRIDE Threat Model*, Microsoft Security Development Lifecycle, 2020.

[7] PyCQA, *Bandit: A Python AST-based Security Linter*, Python Code Quality Authority, 2024.

End of IEEE Technical Report — SecureDrop DevSecOps Security Sprint (Fall 2026)